

# Cidades ESG Inteligentes

## Coleta+ ESG API

Pipeline CI/CD, containerização e deploy automatizado em ambientes de staging e produção

|             |                                                         |
|-------------|---------------------------------------------------------|
| Integrante  | [Seu nome — RM]                                         |
| Stack       | Java 21 · Spring Boot 4 · Oracle 23 Free                |
| DevOps      | GitHub Actions · Docker · Docker Compose · GHCR · Azure |
| Repositório | github.com/justinojosesilva/coleta-plus                 |
| Staging     | http://57.156.71.196:8081/swagger-ui.html               |
| Produção    | http://57.156.71.196:8080/swagger-ui.html               |

# 1. Descrição do pipeline

O Coleta+ é uma API REST de gestão de resíduos em pontos de coleta urbanos: registra resíduos e coletas, gera alertas automáticos quando um ponto atinge 90% da capacidade e calcula indicadores ESG (volume reciclado, pontos sobrecarregados e eficiência de coleta). Nesta fase o projeto passou a ter um ciclo de vida totalmente automatizado, do commit até a produção.

## Ferramentas

| Ferramenta                         | Papel                                                                                                                                           |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| GitHub Actions                     | Orquestra o pipeline ( <code>.github/workflows/ci-cd.yml</code> + workflow reutilizável <code>deploy.yml</code> ).                              |
| Maven + JUnit 5 + Mockito          | Build e testes: Surefire para os unitários e Failsafe para os de integração.                                                                    |
| Oracle 23 Free (service container) | Banco real usado pelo teste de integração dentro do runner.                                                                                     |
| Docker Buildx + GHCR               | Gera a imagem e publica no GitHub Container Registry.                                                                                           |
| GitHub Environments                | <code>staging</code> e <code>production</code> , com secrets próprios; produção exige aprovação manual e só aceita a branch <code>main</code> . |
| Azure Virtual Machine              | Ubuntu 24.04 (Standard_B2as_v2) com Docker; hospeda os dois ambientes isolados.                                                                 |

## Etapas



## Lógica de funcionamento

- Gatilhos:** todo *push* na `main` executa o fluxo completo; *pull requests* executam só build e testes, validando a mudança antes do merge. Também pode ser disparado manualmente.
- Build & testes:** compila com Java 21 e executa `./mvnw verify`. Os testes unitários cobrem regras de negócio (baixa de estoque na coleta, alerta de capacidade de 90%, cálculo de eficiência ESG), geração/validação de JWT e mapeamento de papéis. O teste de integração sobe a aplicação inteira contra um Oracle real, roda as migrations do Flyway e valida com `ddl-auto=validate` que todas as entidades batem com o schema. Relatórios e o `.jar` ficam como artefatos.
- Imagem:** construída uma única vez e marcada com o hash do commit. *Build once, deploy many:* staging e produção recebem exatamente o mesmo artefato.

- **Deploy:** o runner gera o arquivo `<ambiente>.env` a partir dos secrets do environment, copia os arquivos por SSH para a VM e executa `deploy/deploy.sh`, que faz `docker compose pull + up -d` e aguarda o container ficar *healthy*.
- **Smoke test:** após cada deploy, o pipeline chama `/actuator/health`, confere em `/actuator/info` se o ambiente e a versão são os esperados, faz login e consulta os indicadores ESG. Qualquer falha interrompe o fluxo e produção não é liberada.
- **Segurança:** senhas do banco, segredo JWT e chave SSH ficam em GitHub Secrets. Nada sensível é versionado, e o banco de staging/produção não expõe porta para fora da VM.

## 2. Docker

### Arquitetura

**GitHub Actions** → publica `ghcr.io/justinojosesilva/coleta-plus:sha-xxxxxxx` → acessa a VM via SSH

**VM Azure · Ubuntu 24.04 · Docker Engine** (portas públicas: 22, 8080, 8081)

#### Projeto Compose `coleta-staging`

app (Spring Boot) · porta **8081**→8080

oracle-db (Oracle 23 Free) · sem porta pública

rede backend · volume oracle-data

#### Projeto Compose `coleta-production`

app (Spring Boot) · porta **8080**→8080

oracle-db (Oracle 23 Free) · sem porta pública

rede backend · volume oracle-data

Cada ambiente é um projeto Docker Compose independente, com rede, volume e banco próprios. Um deploy em staging nunca afeta os dados de produção. A API encontra o banco pelo nome do serviço (`oracle-db`) na rede interna e só sobe quando o healthcheck do banco passa (`depends_on: service_healthy`).

### Dockerfile (multi-stage)

```
# Estágio 1: build (JDK + Maven), não vai para produção
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /workspace
COPY pom.xml mvnw ./
COPY .mvn .mvn
RUN ./mvnw -B -q dependency:go-offline          # camada de dependências em cache
COPY src src
RUN ./mvnw -B -q -DskipTests package && cp target/coleta-plus-*.jar app.jar

# Estágio 2: runtime (somente JRE)
FROM eclipse-temurin:21-jre
RUN groupadd --system app && useradd --system --gid app --home /app app
WORKDIR /app
COPY --from=build --chown=app:app /workspace/app.jar app.jar
USER app                                # processo sem root
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75 -XX:+ExitOnOutOfMemoryError"
EXPOSE 8080
HEALTHCHECK --interval=15s --timeout=5s --start-period=90s --retries=5 \
  CMD curl -fs http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/app.jar"]
```

### Imagem criada

| Item            | Valor                                                                  |
|-----------------|------------------------------------------------------------------------|
| Registry / nome | ghcr.io/justinojosesilva/coleta-plus                                   |
| Tags            | sha-<commit> (imutável, usada nos deploys) e latest                    |
| Base de runtime | eclipse-temurin:21-jre (Ubuntu)                                        |
| Tamanho         | ~425 MB (só JRE + .jar; o estágio de build com JDK/Maven fica de fora) |
| Usuário         | app (não-root)                                                         |
| Healthcheck     | /actuator/health (inclui conexão com o banco)                          |

## Comandos usados

| Comando                                                                                    | Finalidade                                             |
|--------------------------------------------------------------------------------------------|--------------------------------------------------------|
| cp .env.example .env                                                                       | Criar a configuração local                             |
| docker compose up -d --build                                                               | Compilar a imagem e subir app + banco localmente       |
| docker compose ps / logs -f app                                                            | Ver status (healthy) e logs                            |
| docker compose down -v                                                                     | Derrubar o ambiente e apagar o volume                  |
| docker build -t coleta-plus:local .                                                        | Build manual da imagem                                 |
| docker compose -p coleta-staging -f docker-compose.deploy.yml --env-file staging.env up -d | Deploy de um ambiente na VM (executado pelo deploy.sh) |
| ./infra/create-azure-vm.sh                                                                 | Criar a VM no Azure com Docker (cloud-init)            |

### 3. Pipeline em execução

The screenshot displays the GitHub Actions interface for the repository 'justinojosesilva / coleta-plus'. The 'Actions' tab is selected, showing a workflow named 'fix(deploy): YAML inválido no compose de deploy + validação no CI #5'. The workflow status is 'Success', triggered by a push to the 'main' branch 10 minutes ago. The total duration is 9m 30s, and there are 3 artifacts. The workflow steps are: 'Build & testes' (1m 39s), 'Imagem Docker (GHCN)' (53s), 'Deploy staging / staging' (2m 53s), and 'Deploy produção / production' (1m 54s). The 'Deployment protection rules' section is also visible.

Figura 1: execução completa do workflow CI/CD no GitHub Actions.

```
● ● ● GitHub Actions - job "Build & testes" (run #5, commit 12729d9)
$ docker compose config --quiet # validação dos arquivos Compose: OK
$ ./mvnw -B verify
[INFO] T E S T S
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.609 s -- in br.com.fiap.coleta_plus.security.JwtServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.014 s -- in br.com.fiap.coleta_plus.security.CustomUserDetailsServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.188 s -- in br.com.fiap.coleta_plus.service.IndicatorsServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.146 s -- in br.com.fiap.coleta_plus.service.AlertServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.126 s -- in br.com.fiap.coleta_plus.service.CollectionServiceTest
[INFO] Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
[INFO] Building jar: /home/runner/work/coleta-plus/coleta-plus/target/coleta-plus-0.0.1-SNAPSHOT.jar
[INFO] T E S T S
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 9.963 s -- in br.com.fiap.coleta_plus.ColetaPlusApplicationIT
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 20.599 s
```

Figura 2: build e testes automatizados (unitários + integração com Oracle).

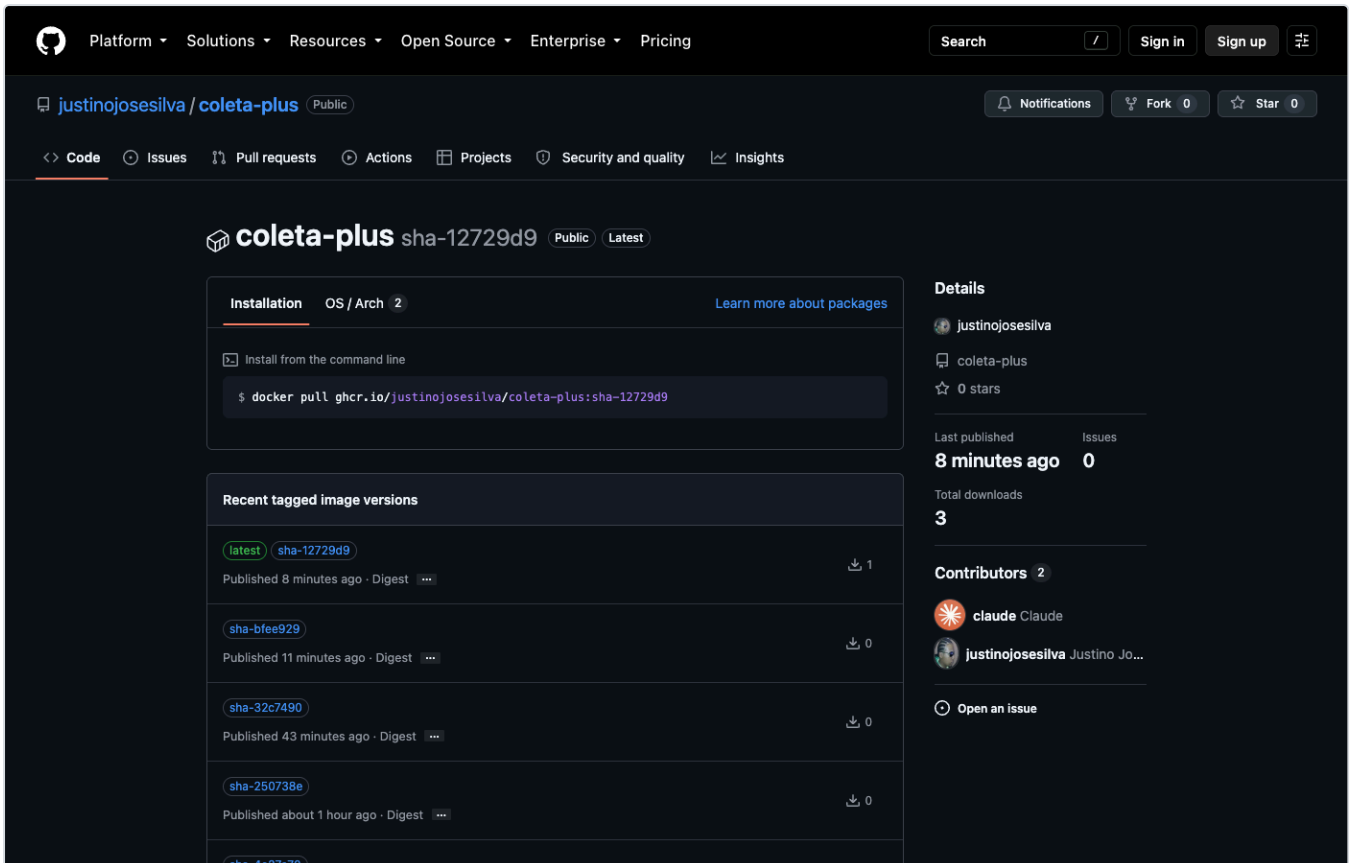


Figura 3: imagem Docker publicada no GitHub Container Registry.

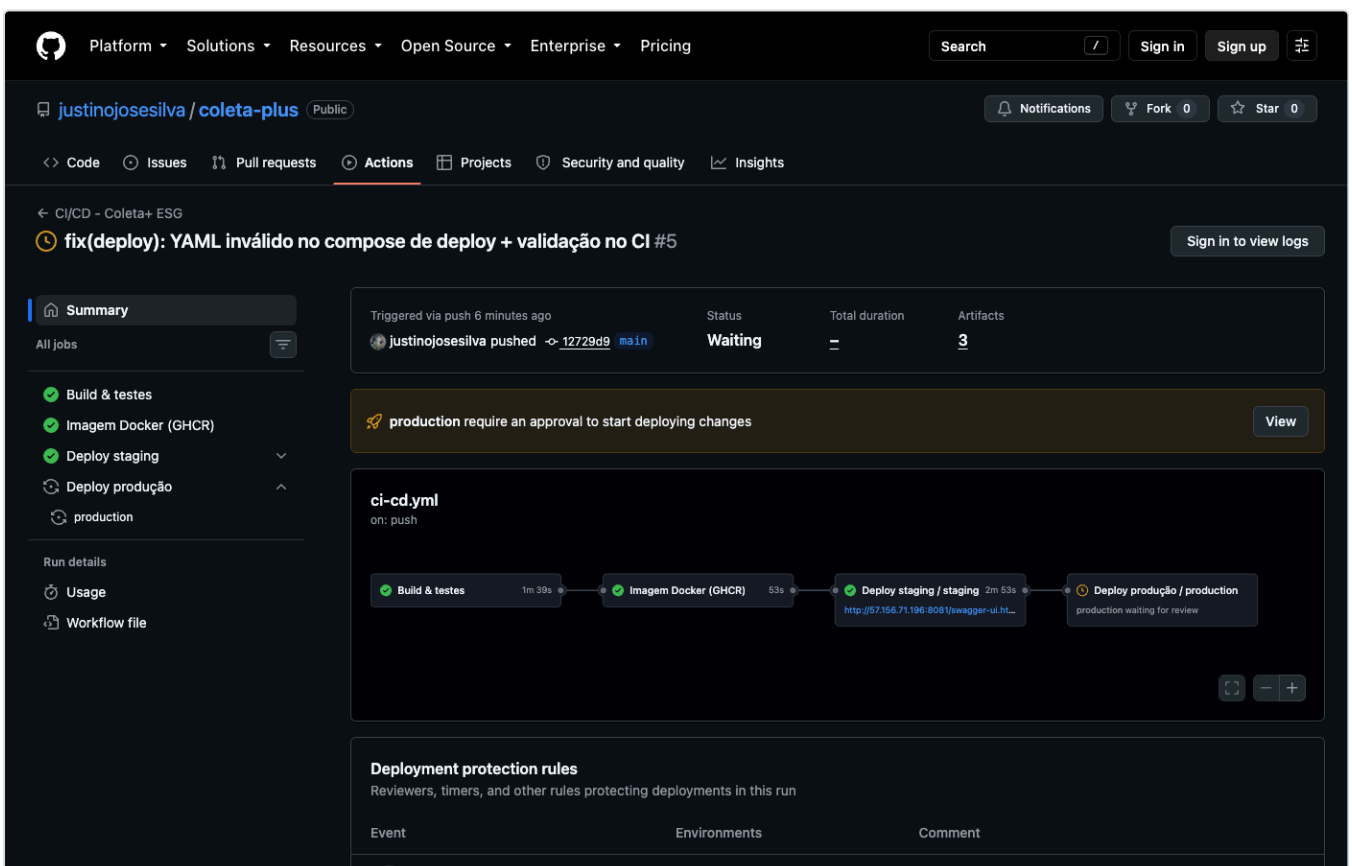


Figura 4: gate de aprovação manual antes do deploy em produção.

# 4. Ambientes de staging e produção

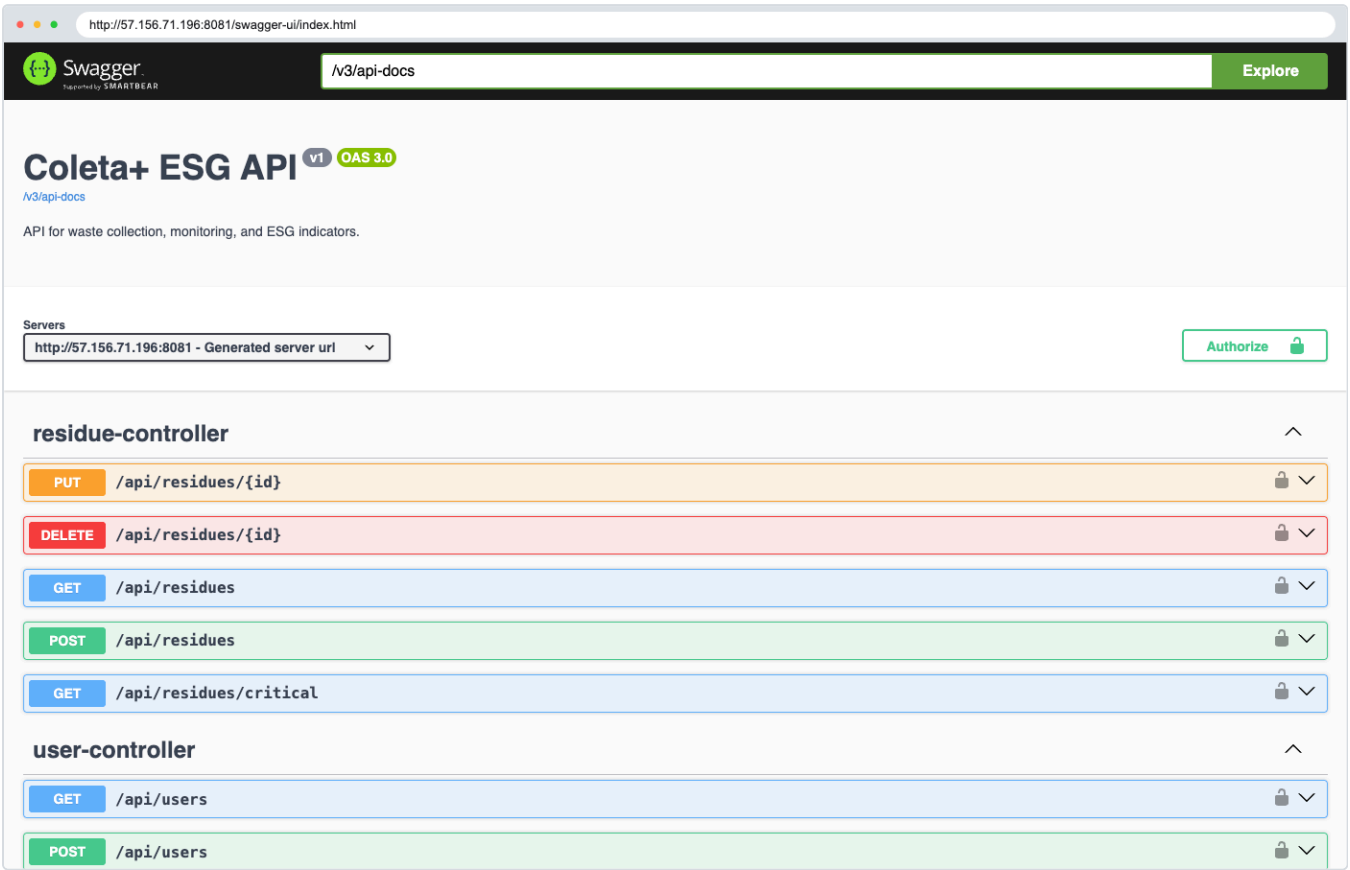


Figura 5: ambiente de **staging** (porta 8081) em funcionamento.

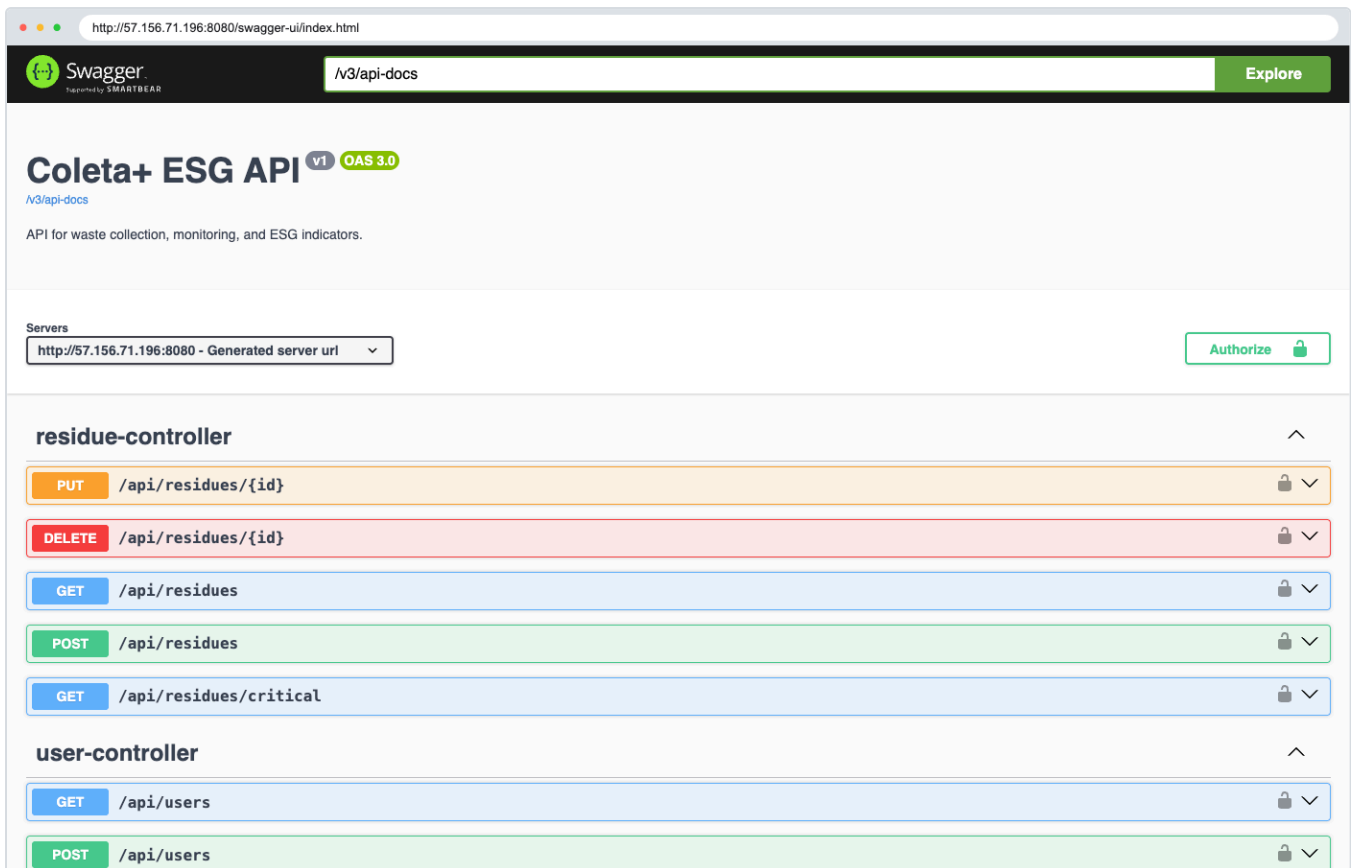


Figura 6: ambiente de **produção** (porta 8080) em funcionamento.

```

● ● ●  ssh azureuser@57.156.71.196 - VM Azure vm-coleta-plus (chilecentral)

azureuser@vm-coleta-plus:~$ docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
NAMES          IMAGE                                STATUS          PORTS
coleta-production-app-1 ghcr.io/justinojosesilva/coleta-plus:sha-12729d9 Up About a minute (healthy) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
coleta-production-oracle-db-1 gvenzl/oracle-free:23-slim-faststart Up About a minute (healthy) 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
coleta-staging-app-1 ghcr.io/justinojosesilva/coleta-plus:sha-12729d9 Up 5 minutes (healthy)
coleta-staging-oracle-db-1 gvenzl/oracle-free:23-slim-faststart Up 5 minutes (healthy)

azureuser@vm-coleta-plus:~$ docker volume ls; docker network ls --filter name=coleta
DRIVER VOLUME NAME
local  coleta-production_oracle-data
local  coleta-staging_oracle-data

NETWORK ID NAME DRIVER SCOPE
c53d957123d0 coleta-production_backend bridge local
05dce7980544 coleta-staging_backend bridge local

azureuser@vm-coleta-plus:~$ curl -s localhost:8081/actuator/info; curl -s localhost:8080/actuator/info
{"app":{"name":"Coleta+ ESG API","environment":"staging","version":"sha-12729d9"}}
{"app":{"name":"Coleta+ ESG API","environment":"production","version":"sha-12729d9"}}

```

Figura 7: containers dos dois ambientes rodando na VM do Azure.

```

● ● ●  coleta-plus - ambiente local (Docker Compose)

$ docker compose ps
NAME          IMAGE                                STATUS          PORTS
coleta-app    coleta-plus:local                  Up 3 minutes (healthy) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
coleta-oracle gvenzl/oracle-free:23-slim-faststart Up 6 minutes (healthy) 0.0.0.0:1521->1521/tcp, [::]:1521->1521/tcp

$ curl -s localhost:8080/actuator/health
{"groups":{"liveness","readiness"},"status":"UP"}

$ curl -s localhost:8080/actuator/info
{"app":{"name":"Coleta+ ESG API","environment":"local","version":"local"}}

$ curl -s -H "Authorization: Bearer $TOKEN" localhost:8080/api/indicadores-sustentabilidade
{"volumeRecycled":0,"volumeNonRecycled":0,"pointsOverCapacity":[],"collectionEfficiency":0.0}

$ docker images coleta-plus:local
REPOSITORY TAG SIZE
coleta-plus local 425MB

```

Figura 8: ambiente local com Docker Compose (app + Oracle, ambos healthy).

## 5. Desafios encontrados e soluções

---

### 1. Todos os endpoints protegidos retornavam 403, inclusive para o administrador

O smoke test do deploy (login + consulta de indicadores) revelou um bug que existia desde a fase anterior: o `CustomUserDetailsService` registrava a permissão como `ADMIN`, mas

`@PreAuthorize("hasRole('ADMIN')")` procura `ROLE_ADMIN`. **Solução:** prefixar `ROLE_` na authority e criar um teste unitário de regressão.

### 2. Entidade apontando para tabela inexistente

Com a autorização corrigida, a consulta aos indicadores ESG falhou com `ORA-00942`: a entidade

`Collection` usava a tabela `collections`, mas a migration cria `TB_COLLECTIONS`. **Solução:** corrigir o mapeamento e fazer o teste de integração rodar com `ddl-auto=validate`. Agora qualquer divergência entre entidades e migrations quebra o pipeline antes do deploy (o teste falha com *missing table*).

### 3. Testes dependiam de um banco Oracle

O único teste existente (`contextLoads`) exigia um Oracle acessível, o que inviabilizava o CI. **Solução:** separar testes unitários (Mockito, sem banco, rodam no `mvn test`) de testes de integração (`*IT.java`, via Failsafe) e usar um *service container* Oracle 23 Free no GitHub Actions.

### 4. Variável de ambiente do segredo JWT ignorada

A propriedade usava `${JWT.SECRET:fiap}`, então a variável `JWT_SECRET` definida no container nunca era lida e todos os ambientes usavam o segredo padrão. **Solução:** corrigir para `${JWT_SECRET:fiap}` e injetar segredos diferentes por ambiente.

### 5. Oracle na nuvem e custo

Serviços PaaS gratuitos (Render, App Service) não oferecem Oracle e têm pouca memória para rodá-lo em container. **Solução:** uma VM `B2as_v2` (8 GB + 4 GB de swap via `cloud-init`) hospedando os dois ambientes isolados por projeto Compose, coberta pelo crédito do Azure for Students e desligável com `az vm deallocate`.

### 6. Saber se o deploy realmente funcionou

`docker compose up` retorna sucesso mesmo que a aplicação caia logo depois. **Solução:** Spring Boot Actuator + `HEALTHCHECK` no Dockerfile; o `deploy.sh` espera o status *healthy* (e imprime os logs se falhar), e o pipeline confirma ambiente/versão via `/actuator/info`.

### 7. Restrições da assinatura Azure for Students e falhas no primeiro deploy

A série B não estava disponível em `eastus` para a assinatura de estudante (apenas regiões permitidas por *policy*). A VM foi criada em `chilecentral` com `Standard_B2as_v2`. Os acentos nos comentários do `cloud-init.yml` corromperam o YAML enviado e o Docker não foi instalado, então o arquivo passou a ser só ASCII. Por fim, o primeiro deploy em staging falhou porque um texto de ajuda (ex.: ...) dentro de `${IMAGE:?...}` quebrava o YAML do compose de deploy. **Solução:** correção da mensagem e uma nova etapa no pipeline que valida os dois arquivos Compose com `docker compose config` antes do build.

## 6. Checklist de entrega

| Item                                                | OK                                  |
|-----------------------------------------------------|-------------------------------------|
| Projeto compactado em .ZIP com estrutura organizada | <input checked="" type="checkbox"/> |
| Dockerfile funcional                                | <input checked="" type="checkbox"/> |
| docker-compose.yml ou arquivos Kubernetes           | <input checked="" type="checkbox"/> |
| Pipeline com etapas de build, teste e deploy        | <input checked="" type="checkbox"/> |
| README.md com instruções e prints                   | <input checked="" type="checkbox"/> |
| Documentação técnica com evidências (PDF ou PPT)    | <input checked="" type="checkbox"/> |
| Deploy realizado nos ambientes staging e produção   | <input checked="" type="checkbox"/> |

### Estrutura do projeto entregue

```
coleta-plus/
├── Dockerfile
├── docker-compose.yml           # ambiente local (app + Oracle)
├── .env.example
├── deploy/
│   ├── docker-compose.deploy.yml # staging e produção (imagem do GHCR)
│   ├── deploy.sh                 # executado na VM pelo pipeline
│   ├── staging.env.example
│   └── production.env.example
├── infra/
│   ├── create-azure-vm.sh        # cria a VM no Azure
│   └── cloud-init.yml            # instala Docker na VM
├── .github/workflows/
│   ├── ci-cd.yml                 # build → testes → imagem → staging → produção
│   └── deploy.yml                 # deploy reutilizável por ambiente
├── src/                          # código-fonte + testes
├── docs/                         # esta documentação + prints
└── README.md
```